Decision and Risk Analysis

Lecture 7: **A Simple Manual Example of MCMC**

Metropolis-Hastings for a Bernoulli Parameter

DAEN 427 / ISEN 427 / ISEN 627 - Fall 2026

Alexander Abuabara



We want to understand how MCMC can generate samples from a posterior distribution.

Suppose we observe four Bernoulli trials:

$$y = (1, 1, 1, 0)$$

That is:

$$3 \text{ successes}, \qquad 1 \text{ failure}$$

The unknown parameter is:

$$\theta = \Pr(y = 1)$$

Prior and Likelihood

Assume a uniform prior:

$$\theta \sim \text{Uniform}(0, 1)$$

The likelihood for 3 successes and 1 failure is:

$$p(y \mid \theta) \propto \theta^3(1-\theta)^1$$

Since the prior is uniform, the posterior is proportional to:

$$p(\theta \mid y) \propto \theta^3(1-\theta)$$

We define the unnormalized posterior density:

$$f(\theta) = \theta^3(1-\theta)$$

Metropolis Rule (Symmetric Proposal)

Start at some current value:

$$\theta_t$$

Propose a new value:

$$\theta^*$$

For a symmetric proposal, $q(\theta^* \mid \theta_t) = q(\theta_t \mid \theta^*)$, compute the acceptance ratio:

$$r = \frac{f(\theta^*)}{f(\theta_t)}$$

(check *why the algorithm works)*

Then compute:

$$\alpha = \min(1, r)$$

Accept the proposed value if:

$$u < \alpha, \qquad u \sim \text{Uniform}(0, 1)$$

Step 1: Move from 0.50 to 0.60

Start at:

$$\theta_0 = 0.50$$

Compute:

$$f(0.50) = 0.50^3(1 - 0.50) = 0.125 \times 0.50 = 0.0625$$

Propose:

$$\theta^* = 0.60$$

Compute:

$$f(0.60) = 0.60^3(1 - 0.60) = 0.216 \times 0.40 = 0.0864$$

Acceptance ratio:

$$r = \frac{0.0864}{0.0625} = 1.3824$$

So:

$$\alpha = 1$$

Accept:

$$0.50 \rightarrow 0.60$$

Step 2: Move from 0.60 to 0.80

Current value:

$$\theta_t = 0.60$$

Propose:

$$\theta^* = 0.80$$

Compute:

$$f(0.80) = 0.80^3(1 - 0.80) = 0.512 \times 0.20 = 0.1024$$

Acceptance ratio:

$$r = \frac{0.1024}{0.0864} = 1.185$$

Thus:

$$\alpha = 1$$

Accept:

$$0.60 \rightarrow 0.80$$

Step 3: A Rejected Proposal

Current value:

$$\theta_t = 0.80$$

Propose:

$$\theta^* = 0.95$$

Compute:

$$f(0.95) = 0.95^3(1 - 0.95) \approx 0.8574 \times 0.05 = 0.0429$$

Acceptance ratio:

$$r = \frac{0.0429}{0.1024} \approx 0.419$$

So:

$$\alpha = 0.419$$

Suppose we draw:

$$u = 0.70$$

Since:

$$0.70 > 0.419$$

reject the proposal and stay at:

$$\theta = 0.80$$

Step 4: Move from 0.80 to 0.70

Current value:

$$\theta_t = 0.80$$

Propose:

$$\theta^* = 0.70$$

Compute:

$$f(0.70) = 0.70^3(1 - 0.70) = 0.343 \times 0.30 = 0.1029$$

Acceptance ratio:

$$r = \frac{0.1029}{0.1024} \approx 1.005$$

Thus:

$$\alpha = 1$$

Accept:

$$0.80 \to 0.70$$

Manual Chain

The chain we recorded is:

$$0.50,\ 0.60,\ 0.80,\ 0.80,\ 0.70$$

Notice the repeated value:

$$0.80,\ 0.80$$

This happens because a rejected proposal keeps the chain at the current state.

After many iterations, the histogram of the chain approximates the posterior:

$$p(\theta \mid y)$$

Summary Table

Step	Current	Proposal	f(Current)	f(Proposal)	α	Result
1	0.50	0.60	0.0625	0.0864	1.000	Accept
2	0.60	0.80	0.0864	0.1024	1.000	Accept
3	0.80	0.95	0.1024	0.0429	0.419	Reject
4	0.80	0.70	0.1024	0.1029	1.000	Accept

Key Idea

MCMC does not only search for the best value.

It constructs a chain whose long-run behavior matches the posterior distribution.

Regions with higher posterior probability are visited more often.

Regions with lower posterior probability are visited less often, but not necessarily never.

That is why MCMC provides uncertainty, not just a point estimate.

In the simple example, the unknown was one parameter:

$$\theta$$

In Bayesian linear regression, the unknowns are usually:

$$(\beta_0, \beta_1, \sigma)$$

MCMC samples from:

$$p(\beta_0, \beta_1, \sigma \mid x, y)$$

Each draw gives one plausible regression model:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

The collection of draws gives the full posterior uncertainty.

Monte Carlo: Direct Sampling

1. Draw N independent samples $x_1, \ldots, x_N$ directly from the target distribution $\pi(x)$.
2. Compute the quantity of interest $g(x_i)$ for every draw.
3. Estimate its expected value by $\frac{1}{N} \sum_{i=1}^{N} g(x_i)$.

Pro: Simple; draws are independent.

Con: Direct sampling from a complicated target can be difficult.

Contrast: Metropolis–Hastings uses dependent Markov-chain draws, avoiding the need to sample directly from π.

Metropolis vs. Metropolis–Hastings

For current state x_t, propose $x^* \sim q(x^* \mid x_t)$ and calculate

$$\alpha = \min\left\{1, \frac{\pi(x^*)\, q(x_t \mid x^*)}{\pi(x_t)\, q(x^* \mid x_t)}\right\}.$$

Draw $u \sim \text{Uniform}(0,1)$. If $u < \alpha$, set $x_{t+1} = x^*$; otherwise set $x_{t+1} = x_t$. Repeat.

- **Metropolis:** symmetric proposal $q(y \mid x) = q(x \mid y)$, so the q terms cancel and $\alpha = \min\{1, \pi(x^*)/\pi(x_t)\}$.
- **Metropolis–Hastings:** also permits asymmetric proposals; keep the q correction.

Our earlier numerical example used the *Metropolis* (symmetric-proposal) acceptance rule.

Independence Sampling

An **independence sampler** is a Metropolis–Hastings algorithm with

$$q(x^* \mid x_t) = q(x^*).$$

- The proposal distribution is **fixed** and does **not** depend on the current chain state.
- Its acceptance probability is

$$\alpha = \min\left\{1, \frac{\pi(x^*)\, q(x_t)}{\pi(x_t)\, q(x^*)}\right\}.$$

- It is a **special case of Metropolis–Hastings**, but **not generally of Metropolis**: independent proposals need not be symmetric.

Important: Independent *proposals* do not imply independent *accepted chain states*.

Homework

1. What are the steps to the Monte Carlo Algorithm?
2. Name one pro and one con for the Monte Carlo Algorithm.
3. What are the steps to the Metropolis-Hastings Algorithm?
4. What's the difference between the Monte Carlo and Metropolis-Hastings algorithms?
5. What's the difference between the Metropolis and Metropolis-Hastings algorithms?
6. Identify whether the below statements about independence sampling are True or False. If False, explain.
   - 6.1 The proposal model depends on the current value of the chain.
   - 6.2 The proposal model is the same every time.
   - 6.3 It is a special case of the Metropolis-Hastings algorithm.
   - 6.4 It is a special case of Metropolis algorithm.